

EC200A-CN(TA) QuecOpen GNSS API 参考手册

LTE Standard 模块系列

版本：1.0.0

日期：2022-08-11

状态：临时文档



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登陆网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬软件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2022，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2022.

文档历史

修订记录

版本	日期	作者	变更表述
-	2022-03-31	Zac ZHANG	文档创建
1.0.0	2022-08-11	Sunshine HUANG	临时版本

目录

文档历史	3
目录	4
表格索引	6
1 引言	7
2 GNSS 功能特性介绍	8
2.1. 功能特性	8
2.2. 性能指标	8
3 调用流程	10
4 GNSS API	12
4.1. 头文件	12
4.2. 函数概览	12
4.3. 函数详解	13
4.3.1. ql_gnss_init	13
4.3.2. ql_gnss_deinit	13
4.3.3. ql_gnss_set_ind_cb	14
4.3.4. ql_gnss_set_service_error_cb	14
4.3.5. ql_gnss_start	15
4.3.6. ql_gnss_stop	15
4.3.7. ql_gnss_set_start_mode	16
4.3.7.1. QL_GNSS_START_MODE_E	16
4.3.8. ql_gnss_get_engine_state	17
4.3.8.1. QL_GNSS_ENGINE_STATE_E	17
4.3.9. ql_gnss_set_constellation	18
4.3.9.1. QL_GNSS_CONSTELLATION_MASK_E	18
4.3.10. ql_gnss_get_constellation	19
4.3.11. ql_gnss_set_nmea_version	19
4.3.11.1. QL_GNSS_NMEA_VERSION_ID_E	19
4.3.12. ql_gnss_get_nmea_version	20
4.3.13. ql_gnss_set_nmea_type	20
4.3.13.1. QL_GNSS_NMEA_TYPE_ID_E	21
4.3.14. ql_gnss_set_agnss_mode	22
4.3.14.1. QL_GNSS_AGNSS_MODE_E	22
4.3.15. ql_gnss_inject_agnss_data	23
4.3.16. ql_gnss_inject_utc_time	23
4.3.17. ql_gnss_suspend	24
4.3.18. ql_gnss_resume	24
4.4. 示例代码	24
5 AGNSS 辅助定位技术	29
5.1. 功能简介	29

5.2. 功能使用	29
5.2.1. 服务器地址解析	29
5.2.2. 账号认证	29
5.2.3. 数据请求	30
5.2.4. 数据回复	31
5.2.5. 数据注入	31
6 附录 参考文档及术语缩写	32

Preliminary Document
Not Checked

表格索引

表 1: 性能指标	8
表 2: 函数概览	12
表 3: 账号认证参数描述	30
表 4: 请求内容字段描述	30
表 5: HTTP 错误码定义	31
表 6: 参考文档	32
表 7: 术语缩写	32

1 引言

移远通信 EC200A-CN(TA)模块支持 QuecOpen®方案；QuecOpen®是开源的基于 Linux 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]。

本文档主要介绍如何通过使用移远通信 EC200A-CN(TA) QuecOpen 模块 SDK 中提供的 GNSS API 实现模块的 GNSS 相关功能。

GNSS 即全球导航卫星系统，泛指所有的卫星导航系统，包括全球卫星导航、区域导航以及增强型卫星导航。全球卫星导航系统有 GPS、GLONASS、BDS、Galileo 等；区域导航系统有日本的 QZSS 及印度的 IRNSS；增强型卫星导航系统有 WAAS、SDCM、EGNOS、MSAS、GAGAN。

2 GNSS 功能特性介绍

移远通信 EC200A-CN(TA) QuecOpen 模块采用 UC6228CI 芯片，支持多个系统的 GNSS，可并行接收和处理 GPS L1、BDS B1、GLONASS L1 和 Galileo E1。同时支持扩展系统 QZSS 和 SBAS。支持 AGNSS 快速定位技术等，使定位更快速、更准确。

本章节主要介绍移远通信 EC200A-CN(TA) QuecOpen 模块的 GNSS 功能特性和性能指标。

2.1. 功能特性

- 支持多星座定位功能，包括 GPS、GLONASS、BDS、Galileo 和 QZSS 等；
- 支持 AGNSS 快速定位技术；
- 支持星基增强系统（SBAS），包括 WAAS、EGNOS、MSAS 和 GAGAN 等；
- 支持精密单点定位和差分定位（DGPS）。

2.2. 性能指标

表 1: 性能指标

指标	性能	备注
水平定位精度（CEP50）	< 2 米	开阔地带； 定位模式：GPS + BDS 模式
高程定位精度（CEP50）	< 3 米	开阔地带； 定位模式：GPS + BDS 模式
速度精度（CEP50）	0.01 米/秒	开阔地带； 定位模式：GPS + BDS 模式
同时跟踪通道数	64	-
冷启动 TTFF	< 28 秒	开阔地带； 定位模式：GPS + BDS 模式
热启动 TTFF	≤ 1 秒	开阔地带； 定位模式：GPS + BDS 模式
重捕获时间	≤ 1 秒	定位模式：GPS + BDS 模式

捕获灵敏度（冷启动）	-147 dBm	定位模式：GPS + BDS 模式
跟踪灵敏度	-160 dBm	定位模式：GPS + BDS 模式
捕获灵敏度（热启动）	-155 dBm	定位模式：GPS + BDS 模式
重捕灵敏度	-158 dBm	定位模式：GPS + BDS 模式

Preliminary Document
Not Checked

3 调用流程

GNSS API 调用的基本流程如下图所示；GNSS API 示例请参考第4.4 章。

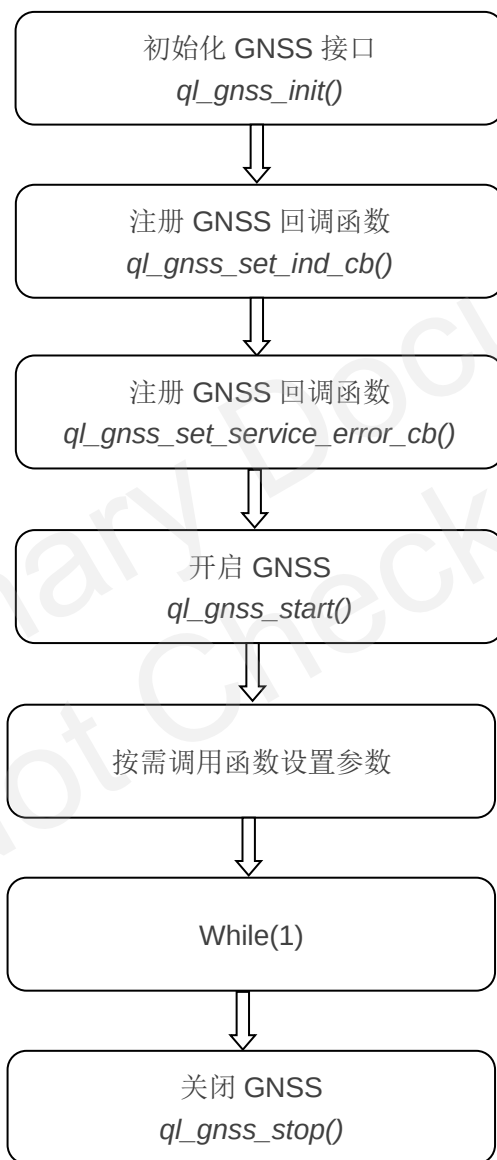


图 1: GNSS API 基本调用流程

备注

1. 不可在回调函数中调用任何其他 GNSS API 函数。
2. 不可在回调函数中处理阻塞任务。
3. 需使能配置文件/etc/ql_locationd.conf 中的 *dynamic_set=1* 方可调用函数设置参数。

Preliminary Document
Not Checked

4 GNSS API

4.1. 头文件

GNSS API 头文件为 `ql_gnss.h`，位于 SDK 包的 `ql-sysroots/usr/include/ql-sdk/ql_gnss.h` 目录下；错误码定义在 `ql_type.h`，位于 SDK 包的 `ql-sysroots/usr/include/ql-sdk/ql_type.h` 目录下。

4.2. 函数概览

表 2：函数概览

函数	说明
<code>ql_gnss_init()</code>	初始化 GNSS 服务
<code>ql_gnss_deinit()</code>	去初始化 GNSS 服务
<code>ql_gnss_set_ind_cb()</code>	注册 GNSS 数据上报回调函数
<code>ql_gnss_start()</code>	开启 GNSS
<code>ql_gnss_stop()</code>	关闭 GNSS
<code>ql_gnss_set_start_mode()</code>	设置 GNSS 启动模式
<code>ql_gnss_get_engine_state()</code>	获取当前 GNSS 引擎状态
<code>ql_gnss_set_constellation()</code>	设置星系组合配置
<code>ql_gnss_get_constellation()</code>	获取当前的星系组合配置
<code>ql_gnss_set_nmea_type()</code>	设置上报 NMEA 语句类型
<code>ql_gnss_get_nmea_version()</code>	获取当前 NMEA 版本配置
<code>ql_gnss_set_nmea_version()</code>	设置 NMEA 版本配置

<code>ql_gnss_set_agnss_mode()</code>	设置 AGNSS 启动模式
<code>ql_gnss_inject_agnss_data()</code>	注入 AGNSS 数据
<code>ql_gnss_inject_utc_time()</code>	注入当前 UTC 时间
<code>ql_gnss_suspend()</code>	休眠 GNSS 引擎
<code>ql_gnss_resume()</code>	唤醒 GNSS 引擎
<code>ql_gnss_set_service_error_cb()</code>	注册 GNSS 服务异常回调函数

4.3. 函数详解

4.3.1. ql_gnss_init

该函数用于初始化 GNSS 服务。

- 函数原型

```
int ql_gnss_init(void)
```

- 参数

无

- 返回值

`QL_ERR_OK` 函数执行成功
其他值 函数执行失败，详情请参考 `ql_type.h`。

备注

调用其他 GNSS API 之前，必须先调用此函数以初始化 GNSS 服务。

4.3.2. ql_gnss_deinit

该函数用于去初始化 GNSS 服务。

- 函数原型

```
int ql_gnss_deinit(void)
```

- 参数

无

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，详情请参考 *ql_type.h*。

备注

完成其他 GNSS API 的调用之后，需调用此函数以释放 GNSS 相关资源。

4.3.3. ql_gnss_set_ind_cb

该函数用于注册 GNSS 数据上报回调函数。

- 函数原型

```
int ql_gnss_set_ind_cb(ql_gnss_ind_cb_f cb)
```

- 参数

cb:

[In] GNSS 数据上报回调函数。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，详情请参考 *ql_type.h*。

4.3.4. ql_gnss_set_service_error_cb

该函数用于注册 GNSS 服务异常上报回调函数。

- 函数原型

```
int ql_gnss_set_service_error_cb(ql_gnss_error_cb_f cb)
```

- 参数

cb:

[In] GNSS 服务异常回调函数。

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，详情请参考 *ql_type.h*。

4.3.5. ql_gnss_start

该函数用于开启 GNSS。

- 函数原型

```
int ql_gnss_start(void)
```

- 参数

无

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，详情请参考 *ql_type.h*。

备注

1. 默认使用配置文件 */etc/ql_locationd.conf* 配置 GNSS 参数设置。如需调用 API 设置 GNSS 参数，需要更改使能配置文件 */etc/ql_locationd.conf* 中的 *dynamic_set=1*，并须先调用此函数开启 GNSS。
2. 用户可根据 IMEI 号查询模块对应的固件版本号。若查询到的固件版本号为 EC200ACNTAR01A01M2G_OCPU 或 EC200ACNTAR01A02M2G_OCPU，则可能会出现执行 *ql_gnss_start()* 函数失败的情况。此时可修改 *ql_locationd.conf* 中的波特率为 9600。若不是上述两个版本号，则不会出现执行失败的情况。

4.3.6. ql_gnss_stop

该函数用于关闭 GNSS。

- 函数原型

```
int ql_gnss_stop(void)
```

- 参数

无

- 返回值

`QL_ERR_OK` 函数执行成功
其他值 函数执行失败，详情请参考 `ql_type.h`。

4.3.7. ql_gnss_set_start_mode

该函数用于设置 GNSS 引擎启动模式。

- 函数原型

```
int ql_gnss_set_start_mode( QL_GNSS_START_MODE_E mask)
```

- 参数

mask:

[In] GNSS 启动模式；详见第4.3.7.1 章。

- 返回值

`QL_ERR_OK` 函数执行成功
其他值 函数执行失败，详情请参考 `ql_type.h`。

4.3.7.1. QL_GNSS_START_MODE_E

GNSS 启动模式枚举定义如下：

```
typedef enum {
    /** Cold Start Gnss*/
    QL_GNSS_COLD_START = 0,
    /** Warm Start Gnss */
    QL_GNSS_WARM_START,
    /** Hot Start Gnss*/
    QL_GNSS_HOT_START,
} QL_GNSS_START_MODE_E
```

- 成员

成员	描述
<code>QL_GNSS_COLD_START</code>	冷启动
<code>QL_GNSS_WARM_START</code>	温启动
<code>QL_GNSS_HOT_START</code>	热启动

4.3.8. ql_gnss_get_engine_state

该函数用于获取 GNSS 引擎状态。

- 函数原型

```
int ql_gnss_get_engine_state( QL_GNSS_ENGINE_STATE_E * state )
```

- 参数

state:

[Out] GNSS 引擎状态；详见第4.3.8.1章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，详情请参考 ql_type.h。

4.3.8.1. QL_GNSS_ENGINE_STATE_E

GNSS 引擎状态枚举定义如下：

```
typedef enum
{
    /** gnss engine is off */
    QL_GNSS_ENGINE_STATE_OFF = 0,
    /** gnss engine is on */
    QL_GNSS_ENGINE_STATE_ON = 1,
}QL_GNSS_ENGINE_STATE_E
```

- 成员

成员	描述
QL_GNSS_ENGINE_STATE_ON	开启
QL_GNSS_ENGINE_STATE_OFF	关闭

4.3.9. ql_gnss_set_constellation

该函数用于设置星系组合配置，即参与定位的星座。

- 函数原型

```
int ql_gnss_set_constellation ( QL_GNSS_CONSTELLATION_MASK_E mask)
```

- 参数

mask:

[In] 星系组合配置。详见第4.3.9.1章

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，详情请参考 *ql_type.h*。

4.3.9.1. QL_GNSS_CONSTELLATION_MASK_E

星系组合配置枚举定义如下。

```
typedef enum
{
    GPS_SBAS_QZSS           = 0x08,
    BDS_ONLY                = 0x02,
    GPS_BDS_GALILEO_SBAS_QZSS = 0x11,
}QL_GNSS_CONSTELLATION_MASK_E
```

- 成员

成员	描述
<i>GPS_SBAS_QZSS</i>	GPS + SBAS + QZSS
<i>BDS_ONLY</i>	仅 BDS
<i>GPS_BDS_GALILEO_SBAS_QZSS</i>	GPS + BDS + GAL + SBAS + QZSS

4.3.10. ql_gnss_get_constellation

函数用于获取当前的星系组合配置。

- 函数原型

```
int ql_gnss_get_constellation( QL_GNSS_CONSTELLATION_MASK_E * mask)
```

- 参数

mask:

[Out] 获取的星系组合配置。详见第4.3.9.1章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，详情请参考 *ql_type.h*。

4.3.11. ql_gnss_set_nmea_version

该函数用于设置 NMEA 语句版本。

- 函数原型

```
int ql_gnss_set_nmea_version( QL_GNSS_NMEA_VERSION_ID_E version)
```

- 参数

version:

[In] NMEA 语句版本；详见第4.3.11.1章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，详情请参考 *ql_type.h*。

4.3.11.1. QL_GNSS_NMEA_VERSION_ID_E

NMEA 语句版本掩码枚举定义如下：

```
typedef enum {
    QL_GNSS_NMEA_VERSION_V30,      /* NMEA 3.0 */
    QL_GNSS_NMEA_VERSION_V41      /* NMEA 4.1 */
} QL_GNSS_NMEA_VERSION_ID_E
```

- 成员

成员	描述
<code>QL_GNSS_NMEA_VERSION_V30</code>	3.0 版本
<code>QL_GNSS_NMEA_VERSION_V41</code>	4.1 版本

4.3.12. ql_gnss_get_nmea_version

该函数用于获取当前 NMEA 语句版本。

- 函数原型

```
int ql_gnss_get_nmea_version( QL_GNSS_NMEA_VERSION_ID_E* version)
```

- 参数

version:

[Out] 获取的当前 NMEA 语句版本。

- 返回值

`QL_ERR_OK` 函数执行成功
其他值 函数执行失败，详情请参考 *ql_type.h*。

4.3.13. ql_gnss_set_nmea_type

该函数用于设置 NMEA 语句类型配置。

- 函数原型

```
int ql_gnss_set_nmea_type(uint32_t mask)
```

- 参数

mask:

[In] NMEA 类型，用户可自由组合；详见第 4.3.13.1 章。

- 返回值

`QL_ERR_OK` 函数执行成功
其他值 函数执行失败，详情请参考 *ql_type.h*。

4.3.13.1. QL_GNSS_NMEA_TYPE_ID_E

NMEA 类型掩码枚举定义如下：

```
typedef enum {
    QL_NMEA_TYPE_BITMASK_GGA      = (1<<0),
    QL_NMEA_TYPE_BITMASK_GLL      = (1<<1),
    QL_NMEA_TYPE_BITMASK_GSA      = (1<<2),
    QL_NMEA_TYPE_BITMASK_GSV      = (1<<3),
    QL_NMEA_TYPE_BITMASK_RMC      = (1<<4),
    QL_NMEA_TYPE_BITMASK_VTG      = (1<<5),
    QL_NMEA_TYPE_BITMASK_ZDA      = (1<<6),
    QL_NMEA_TYPE_BITMASK_GST      = (1<<7)
} QL_GNSS_NMEA_TYPE_ID_E
```

● 成员

成员	描述
QL_NMEA_TYPE_BITMASK_GGA	输出 GGA 语句
QL_NMEA_TYPE_BITMASK_GLL	输出 GLL 语句
QL_NMEA_TYPE_BITMASK_GSA	输出 GSA 语句
QL_NMEA_TYPE_BITMASK_GSV	输出 GSV 语句
QL_NMEA_TYPE_BITMASK_RMC	输出 RMC 语句
QL_NMEA_TYPE_BITMASK_VTG	输出 VTG 语句
QL_NMEA_TYPE_BITMASK_ZDA	输出 ZDA 语句
QL_NMEA_TYPE_BITMASK_GST	输出 GST 语句

备注

若设置相应 bit 为 1，则表示设置输出对应的 NMEA 语句；若设置为 0 则不输出。

4.3.14. ql_gnss_set_agnss_mode

该函数用于设置 AGNSS 的注入方式。

- 函数原型

```
int ql_gnss_set_agnss_mode(uint32_t mask)
```

- 参数

mask:

[In] AGNSS 的星系组合模式；详见第 4.3.14.1 章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，详情请参考 *ql_type.h*。

4.3.14.1. QL_GNSS_AGNSS_MODE_E

星系组合模式枚举定义如下：

```
typedef enum {
    QL_AGNSS_BITMASK_GPS           = (1<<0),
    QL_AGNSS_BITMASK_BD           = (1<<1),
    QL_AGNSS_BITMASK_GLONASS      = (1<<2),
    QL_AGNSS_BITMASK_GALILEO      = (1<<3),
    QL_AGNSS_BITMASK_QZSS         = (1<<4)
} QL_GNSS_AGNSS_MODE_E
```

- 成员

成员	描述
<i>QL_AGNSS_BITMASK_GPS</i>	注入 GPS 星历数据
<i>QL_AGNSS_BITMASK_BD</i>	注入 BDS 星历数据
<i>QL_AGNSS_BITMASK_GLONASS</i>	注入 GLONASS 星历数据
<i>QL_AGNSS_BITMASK_GALILEO</i>	注入 GALILEO 星历数据
<i>QL_AGNSS_BITMASK_QZSS</i>	注入 QZSS 星历数据

备注

若设置相应 bit 为 1，则表示注入对应的星历数据；若设置为 0 不注入。

4.3.15. ql_gnss_inject_agnss_data

该函数用于注入 AGNSS 数据。

- 函数原型

```
int ql_gnss_inject_agnss_data(curl_rsp_msg_t data)
```

- 参数

data:

[In] AGNSS 数据。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，详情请参考 *ql_type.h*。

4.3.16. ql_gnss_inject_utc_time

该函数用于注入 UTC 时间戳。

- 函数原型

```
int ql_gnss_inject_utc_time(uint64_t timestamp)
```

- 参数

timestamp:

[In] 注入的 UTC 时间戳。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，详情请参考 *ql_type.h*。

4.3.17. ql_gnss_suspend

该函数用于休眠 GNSS 引擎。

- 函数原型

```
int ql_gnss_suspend(void)
```

- 参数

无

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，详情请参考 *ql_type.h*。

4.3.18. ql_gnss_resume

该函数用于唤醒 GNSS 引擎。

- 函数原型

```
int ql_gnss_resume(void)
```

- 参数

无

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，详情请参考 *ql_type.h*。

4.4. 示例代码

本章所述代码示例均摘自 *ql-ol-sdk/sample/gnss*。

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>
#include "test_utils.h"
```

```
#include "ql-sdk/ql_type.h"
#include "ql-sdk/ql_gnss.h"

static void gnss_ind_cb(void *msg)
{
    uint8_t *msg_id = NULL;
    msg_id = msg;
    if(*msg_id == QL_GNSS_NMEA_MSG)
    {
        nmea_srv_ind_msg *data = (nmea_srv_ind_msg *)msg;
        printf("time=%d %s\n",data->time,data->nmea_sentence);
    }
    else if (*msg_id == QL_GNSS_STATUS_MSG)
    {
        gnss_status_ind_msg *data = (gnss_status_ind_msg *)msg;
        switch(data->g_status)
        {
            case QL_GNSS_STATUS_FOTA_START:
                printf("gnss engine fota req\n");
                break;
            case QL_GNSS_STATUS_FOTA_BOOT:
                printf("gnss engine fota bootloader\n");
                break;
            case QL_GNSS_STATUS_FOTA_FIRM:
                printf("gnss engine fota firmware\n");
                break;
            case QL_GNSS_STATUS_WORKING:
                printf("gnss engine working\n");
                break;
            default:
                break;
        };
    }
    else
    {
        printf("Invalue msg !!!");
    }
}

static void gnss_service_error_cb(int error)
{
    printf("==== GNSS Service Abort =====\n");
}
```

```
int main(int argc, char *argv[])
{
    int err = QL_ERR_OK;
    uint32_t mask = 0;
    int input = 0;

    err = ql_gnss_init();
    if(err == QL_ERR_OK)
    {
        printf("ql_gnss_init Successful\n");
    }
    else
    {
        printf("Failed to ql_gnss_init, err=%d\n", err);
    }
    err = ql_gnss_set_ind_cb(gnss_ind_cb);
    if(err != QL_ERR_OK)
    {
        printf("Failed to ql_gnss_set_ind_cb, err=%d\n", err);
    }
    else
    {
        printf("Successful\n");
    }
    err = ql_gnss_set_service_error_cb(gnss_service_error_cb);
    if(err == QL_ERR_OK)
    {
        printf("Successful\n");
    }
    else
    {
        printf("Failed to item_ql_gnss_set_service_error_cb, err=%d\n", err);
    }
    err = ql_gnss_start();
    if(err != QL_ERR_OK)
    {
        printf("Failed to start gnss , err=%d\n", err);
    }
    else
    {
        printf("ql_gnss_start Successful\n");
    }
    sleep(2);
}
```

```
err = ql_gnss_get_engine_state(&mask);
if(err != QL_ERR_OK)
{
    printf("Get engine state   faild %d\n",err);
}
else
{
    if(mask)
    {
        printf("Engine state is on\n");
    }
    else
    {
        printf("Engine state is off\n");
    }
}
err = ql_gnss_get_nmea_version(&mask);
if(err != QL_ERR_OK)
{
    printf("Failed to get nmea_version with version:%d, err=%d\n",mask, err);
}
else
{
    printf("Successful\n");
    if(QL_GNSS_NMEA_VERSION_V30 == mask)
    {
        printf("Nmea_version is 3.0\n");
    }
    else if(QL_GNSS_NMEA_VERSION_V41 == mask)
    {
        printf("Nmea_version is 4.0\n");
    }
}
while(1)
{
    scanf("%d",&input);
    if(input == -1)
    {
        err = ql_gnss_stop();
        if(err != QL_ERR_OK)
        {
            printf("Failed to stop gnss, err=%d\n", err);
        }
    }
    else
```

```
    {  
        printf(" ql_gnss_stop  Successful\n");  
    }  
    err = ql_gnss_deinit();  
    if(err != QL_ERR_OK)  
    {  
        printf("Failed to ql_gnss_deinit, err=%d\n", err);  
    }  
    else  
    {  
        printf(" ql_gnss_deinit  Successful\n");  
    }  
    break;  
}  
}  
  
return 0;  
}
```

5 AGNSS 辅助定位技术

5.1. 功能简介

EC200A-CN(TA)模块支持 AGNSS 辅助定位技术。该服务可为全球提供 GPS、BDS、Galileo、GLONASS、QZSS 五个系统的实时星历，通过 HTTP 协议传输星历信息。

5.2. 功能使用

AGNSS 功能需通过互联网访问服务器，所以设备需能访问互联网。访问 AGNSS 服务器获取星历数据需要使用 AGNSS 账号。

5.2.1. 服务器地址解析

每个使用 AGNSS 功能的用户会被分配一个二级域名（URL），例如：unicore-api1.rx-networks.cn。

为保证全球范围内的服务质量，AGNSS 服务器使用动态负载平衡技术。域名指向的 IP 是不固定的，所以不可直接通过 IP 地址访问服务器。需先通过 DNS 解析到 IP 地址，再使用 IP 地址访问服务器。

5.2.2. 账号认证

AGNSS 服务通过 HTTP 协议提供星历数据，在 HTTP 请求中必须包含账号信息。HTTP 请求中认证字段的格式如下：

```
Authorization: RXN - BASIC cld = ,mld = ,dld = ,pw =
```

参数描述如下：

表 3：账号认证参数描述

参数名	账号示例	描述	备注
<i>cld</i>	ID	用户名	-
<i>mlid</i>	mlid	用户标识	-
<i>dld</i>	-	设备 ID	每个设备 ID 唯一，长度 50 字节以内，为保证服务质量，每个 ID 每天访问 12 次以后可能会被拒绝
<i>pw</i>	Base64 Password	密码	-

5.2.3. 数据请求

数据请求的格式需符合 HTTP 协议。一个完整的请求包含 HTTP 头和请求内容（Content）两部分。按照 HTTP 协议规定，在两部分之间必须有一个空行。请求内容部分符合 JSON 数据格式。以向服务器请求 GPS 和 BDS 的星历数据为例，请求的格式如下所示：

```
POST /rxn-api/locationApi/rbcm HTTP/1.1
Host: XXXXXXXX
Authorization: RXN-BASIC cld=XXXXXXX,mlid=XXXXXX,dld=XXXXXXXXXXXX,pw=XXXXXXX
Content-type: application/json
Accept: application/octet-stream
Content-length: XX

[{"rtAssistance":{"format":"rtcm","msgs":["GPS:1NAF","BDS:2NAF"]}]
```

HTTP 协议头中的 Host 字段为服务器的域名（URL）。请求内容的长度必须与 HTTP 头中的 Content-length 字段相符。各字段描述如下：

表 4：请求内容字段描述

字段	描述
<i>rtAssistance</i>	表示请求的实时星历数据。此字段为固定值，不可更改。
<i>"format":"rtcm"</i>	表示请求的星历数据的格式为 RTCM 格式。此字段为固定值，不可更改。
<i>"msgs":["GPS:1NAF","BDS:2NAF"]</i>	表示请求的是 GPS 和 BDS 的星历数据。此字段可由用户根据需要自由组合，服务器支持的各星座的标识如下： GPS: 1NAF 代表 GPS 星历 BDS: 2NAF 代表 BDS 星历 GLO: 2NAF 代表 GLONASS 星历 GAL: 2NAF 代表 Galileo 星历 QZS: 2NAF 代表 QZSS 星历

例如需要请求 GPS+GAL+BDS 的星座星历，则应为：
"msgs":["GPS:1NAF","GAL:2NAF","BDS:2NAF"]

5.2.4. 数据回复

AGNSS 服务器收到一个合法的请求后会返回符合 HTTP 协议格式的数据，内容包含 HTTP 协议头和星历数据两部分。按照 HTTP 协议规定，两部分之间必须有一个空行。示例如下：

```
HTTP/1.1 200 OK
Content-Type: application/octet-stream
Date: Tue, 06 Nov 2018 08:49:10 GMT
Content-Length: XXXX
Connection: keep-alive

data...
```

HTTP 协议头的第一行中的“200”是 HTTP 的错误码。错误码定义如下：

表 5: HTTP 错误码定义

HTTP 错误码	含义	解释
200	正常	未出现任何异常时返回此值
204	没有数据	服务器接收到合法请求，但无合法星历数据时返回此值
400	非法请求	请求不合法时返回此值
401	认证不通过	当服务器检测到用户名密码等异常时返回此值
500	服务器异常	当服务器发生异常时返回此值
501	暂停服务	当 HTTP 服务暂时不可用时返回此值

5.2.5. 数据注入

接收到服务器返回的数据后，只需依据 HTTP 头和数据之间的空行将 HTTP 头去掉，将其余辅助数据直接通过串口发送给定位芯片或模块即可。

备注

1. 使用 AGNSS 前，请确保已正常拨号，并能正常连接网络。
2. 移远通信仅提供 `ql_gnss_inject_agnss_data()` 用于开启 AGPS，AGNSS 的账号和密码已经包含在 SDK 中，无需额外输入。

6 附录 参考文档及术语缩写

表 6: 参考文档

文档名称
[1] Quectel_EC200A-CN(TA)_QuecOpen_快速开发指导

表 7: 术语缩写

缩写	英文全称	中文全称
AGNSS	Assisted GNSS	辅助 GNSS 技术
API	Application Programming Interface	应用程序接口
BDS	BeiDou Navigation Satellite System	北斗导航系统
DNS	Domain Name Server	域名系统（服务）协议
EGNOS	European Geostationary Navigation Overlay Service	欧洲地球静止导航重叠服务
GAGAN	GPS Aided Geo Augmented Navigation	GPS 辅助型静地轨道增强导航
Galileo	Galileo satellite navigation system	伽利略导航系统
GLONASS	GLOBAL NAVIGATION Satellite System (Russia)	格洛纳斯导航定位系统
GNSS	Global Navigation Satellite Systems	全球导航卫星系统
GPS	Global Positioning System	全球定位系统
HTTP	Hypertext Transfer Protocol	超文本传输协议
IoT	Internet of Things	物联网
IRNSS	Indian Regional Navigation Satellite System	印度区域导航卫星系统
MSAS	Multi-functional Satellite Augmentation System (Japan)	多功能卫星增强系统（日本）

NMEA	NMEA (National Marine Electronics Association) 0183 Interface Standard	NMEA（美国国家海洋电子协会）0183 接口标准
QZSS	Quasi-Zenith Satellite System	准天顶卫星系统
SBAS	Satellite-Based Augmentation System	星基增强系统
SDCM	System of Differential Correction and Monitoring	差分校正和监测系统
SDK	Software Development Kit	软件开发工具包
TTFF	Time To First Fix	次定位时间
URL	Uniform Resource Locator	统一资源定位符
UTC	Universal Time Coordinated	通用协调时
WAAS	Wide Area Augmentation System	广域增强系统